

Chapter 3

WORKING WITH FORMS & USER INPUT IN PHP

3.1. Handling Form Data in PHP

Forms are an essential part of web applications as they allow users to interact with the server by submitting data. PHP provides several ways to handle form data efficiently.

PHP uses the **superglobal** arrays `$_GET` and `$_POST` to retrieve data submitted through forms. PHP superglobal variables are built-in global arrays in PHP that are always accessible, regardless of scope. These variables are available throughout the script and can be accessed directly without the need to declare them as global. Here are the most commonly used PHP superglobal variables

Superglobal Variable	Purpose	Example
<code>\$_GET</code>	Stores data sent via the HTTP GET method (e.g., from URL query strings).	<code>echo \$_GET['name'];</code> (Outputs the value of 'name' from the URL)
<code>\$_POST</code>	Stores data sent via the HTTP POST method (commonly from form submissions).	<code>echo \$_POST['username'];</code> (Outputs the value of 'username' from a form submission)
<code>\$_REQUEST</code>	Contains data from <code>\$_GET</code> , <code>\$_POST</code> , and <code>\$_COOKIE</code> . Can be used to access data from all three.	<code>echo \$_REQUEST['email'];</code> (Can fetch 'email' from GET, POST, or COOKIE)
<code>\$_SERVER</code>	Contains information about the server environment and request headers.	<code>echo \$_SERVER['HTTP_USER_AGENT'];</code> (Outputs the browser's user agent string)
<code>\$_FILES</code>	Holds information about file uploads.	<code>echo \$_FILES['file']['name'];</code> (Outputs the name of the uploaded file)
<code>\$_ENV</code>	Stores environment variables set by the server or system environment.	<code>echo \$_ENV['PATH'];</code> (Outputs the system's PATH environment variable)
<code>\$_COOKIE</code>	Contains values sent by the client via cookies.	<code>echo \$_COOKIE['user'];</code> (Outputs the value of the 'user' cookie)
<code>\$_SESSION</code>	Used to store session variables that persist across multiple pages.	<code>\$_SESSION['username'] = 'JohnDoe';</code> <code>echo \$_SESSION['username'];</code> (Outputs 'JohnDoe')
<code>\$_GLOBALS</code>	A PHP array containing all global variables in the script.	<code>\$x = 10;</code> <code>echo \$GLOBALS['x'];</code> (Outputs 10)

Using \$_GET

- The \$_GET array is used to collect form data sent with the HTTP GET method.
- Data is appended to the URL as query parameters.
- It is suitable for retrieving data but not for sending sensitive information.

Example:

```
<form action="process.php" method="GET">
    Name: <input type="text" name="name">
    <input type="submit" value="Submit">
</form>
```

```
// process.php
if (isset($_GET['name'])) {
    echo "Hello, " . htmlspecialchars($_GET['name']);
}
```

Using \$_POST

- The \$_POST array is used to collect form data sent with the HTTP POST method.
- Data is not visible in the URL, making it more secure for handling sensitive information.

Example:

```
<form action="process.php" method="POST">
    Name: <input type="text" name="name">
    <input type="submit" value="Submit">
</form>
```

```
// process.php
if (isset($_POST['name'])) {
    echo "Hello, " . htmlspecialchars($_POST['name']);
}
```

Example: Simple Feedback Form Processing

A feedback form typically requires multiple input fields, such as name, email, and message. Here's an example:

```
<form action="feedback.php" method="POST">
    Name: <input type="text" name="name" required><br>
    Email: <input type="email" name="email" required><br>
    Message: <textarea name="message" required></textarea><br>
    <input type="submit" value="Send Feedback">
</form>
```

```
// feedback.php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = htmlspecialchars($_POST['name']);
    $email = filter_var($_POST['email'], FILTER_SANITIZE_EMAIL);
    $message = htmlspecialchars($_POST['message']);

    if (!empty($name) && !empty($email) && !empty($message)) {
        echo "Thank you, $name. Your feedback has been received.";
    } else {
        echo "Please fill in all fields.";
    }
}
```

Using \$_REQUEST

- The \$_REQUEST superglobal contains data from both \$_GET and \$_POST.
- It can be convenient for retrieving form data without knowing the request method.

Example:

```
if (isset($_REQUEST['name'])) {
    echo "Hello, " . htmlspecialchars($_REQUEST['name']);
}
```

When to Avoid Using \$_REQUEST

- It is generally discouraged due to security concerns.
- It makes it harder to differentiate between GET and POST data, increasing the risk of unintended data exposure.
- Using \$_REQUEST can lead to vulnerabilities if not properly handled.

3.2. GET vs. POST

Key Differences Between GET and POST

When handling form data in PHP, `GET` and `POST` are the two most commonly used HTTP request methods. Understanding their differences is crucial for writing secure and efficient web applications.

Feature	GET	POST
Visibility	Data is appended to the URL (e.g., <code>example.com?name=John</code>) and visible in the browser's address bar.	Data is sent in the request body and is not visible in the URL.
Security	Less secure since data can be stored in browser history, logs, and bookmarks.	More secure as data is not stored in URLs or browser history.
Request Limits	Limited by the URL length (generally 2048 characters in most browsers).	No significant size limit, suitable for large data submissions.
Caching	Can be cached by browsers and stored in logs.	Not cached by browsers.
Use in Bookmarking	URLs with parameters can be bookmarked and shared.	Cannot be bookmarked or shared.
Data Modification	Used for retrieving data without modifying server state.	Used for modifying server-side data (e.g., submitting forms, updating records).

When to Use GET

- When retrieving non-sensitive data (e.g., search queries, filters, pagination).
- When you want the request to be bookmarkable and shareable.
- When you need better performance with browser caching.

Example: Search Form Using GET

```
<form action="search.php" method="GET">
  <input type="text" name="query" placeholder="Search...">
  <input type="submit" value="Search">
</form>
```

URL after submission:

```
search.php?query=PHP+Tutorial
```

When to Use POST

- When submitting sensitive information (e.g., passwords, personal data).
- When sending large amounts of data, such as file uploads.
- When making database modifications (e.g., inserting, updating, or deleting records).

Example: Login Form Using POST

```
<form action="login.php" method="POST">
  <input type="text" name="username" placeholder="Username">
  <input type="password" name="password" placeholder="Password">
  <input type="submit" value="Login">
</form>
```

POST request does not expose data in the URL, **making it safer for handling credentials.**

Example: Search Form (GET) vs. Login Form (POST)

Use Case	Method	Reason
Search Form	GET	Allows users to bookmark and share queries.
Login Form	POST	Hides credentials from the URL, ensuring better security.

3.3. Validating & Sanitizing User Input

Why Input Validation is Critical

Input validation is an essential process in web development that ensures user-provided data meets the expected criteria. It serves several key purposes:

1. **Security:** Prevents security vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), and Remote Code Execution.
2. **Preventing Errors:** Ensures that data is correctly formatted before processing, reducing unexpected application failures.
3. **Data Integrity:** Helps maintain consistency in stored data, preventing invalid or harmful entries.
4. **User Experience:** Guides users to provide correct input, reducing frustration and improving usability.

PHP Validation Functions

PHP provides several built-in functions to validate and sanitize user input effectively:

- `filter_var()`: Used for validating and sanitizing different types of input.
- `isset()`: Checks if a variable is set and is not null.
- `empty()`: Determines if a variable is empty.
- `trim()`: Removes whitespace from the beginning and end of a string.
- `strlen()`: Returns the length of a string, useful for checking minimum and maximum lengths.

Using `filter_var()` for Validation and Sanitization

The `filter_var()` function is a powerful tool in PHP for both validating and sanitizing user input. It accepts an input variable and a filter type, returning the filtered value or `false` if validation fails.

Validating an Email Address: Validates whether the provided string is a correctly formatted email address.

```
$email = "example@domain.com";
if (filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "Valid email address.";
} else {
    echo "Invalid email address.";
}
```

Validating an Integer: Ensures the input is a valid integer.

```
$age = "25";
if (filter_var($age, FILTER_VALIDATE_INT)) {
    echo "Valid integer.";
} else {
    echo "Invalid integer.";
}
```

Validating a URL: Checks if the input is a properly formatted URL.

```
$url = "https://www.example.com";
if (filter_var($url, FILTER_VALIDATE_URL)) {
    echo "Valid URL.";
} else {
    echo "Invalid URL.";
}
```

Sanitizing a String: Removes HTML tags and special characters that could be used for XSS attacks.

```
$unsafe_string = "<script>alert('Hacked!');</script>";
$safe_string = filter_var($unsafe_string, FILTER_SANITIZE_STRING);
echo $safe_string;
```

Sanitizing an Email: Cleans an email input by removing invalid characters.

```
$unsafe_email = "user@@example.com";
$safe_email = filter_var($unsafe_email, FILTER_SANITIZE_EMAIL);
echo $safe_email;
```

Sanitizing a URL: Removes characters not allowed in a URL.

```
$unsafe_url = "https://example.com/?search=<script>";  
$safe_url = filter_var($unsafe_url, FILTER_SANITIZE_URL);  
echo $safe_url;
```

Common Validation Cases

Required Fields: Ensuring that essential fields are not left empty.

```
if (empty($_POST['username'])) {  
    echo "Username is required.";  
}
```

Valid Email Format: Using `filter_var()` to check if an email address is valid.

```
$email = $_POST['email'];  
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {  
    echo "Invalid email format.";  
}
```

Numeric Input Validation: Ensuring a value is numeric using `is_numeric()`.

```
$age = $_POST['age'];  
if (!is_numeric($age)) {  
    echo "Age must be a number.";  
}
```

Password Length Check: Ensuring that a password meets a minimum length requirement.

```
$password = $_POST['password'];  
if (strlen($password) < 8) {  
    echo "Password must be at least 8 characters long.";  
}
```

Here is a simple example of validating a user registration form in PHP:

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {  
    $username = trim($_POST['username']);  
    $email = trim($_POST['email']);  
    $password = $_POST['password'];  
    $age = $_POST['age'];  
  
    $errors = [];
```

```

if (empty($username)) {
    $errors[] = "Username is required.";
}

if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors[] = "Invalid email format.";
}

if (!is_numeric($age)) {
    $errors[] = "Age must be a number.";
}

if (strlen($password) < 8) {
    $errors[] = "Password must be at least 8 characters long.";
}

if (empty($errors)) {
    echo "Registration successful.";
} else {
    foreach ($errors as $error) {
        echo "<p>$error</p>";
    }
}
}

```

3.4. File Uploads in PHP

Understanding \$_FILES Array

When a user uploads a file using an HTML form with `enctype="multipart/form-data"`, PHP stores information about the uploaded file in the `$_FILES` superglobal array. This array contains:

Key	Description
<code>\$_FILES['file']['name']</code>	Original filename
<code>\$_FILES['file']['type']</code>	MIME type of the file
<code>\$_FILES['file']['tmp_name']</code>	Temporary location where the file is stored
<code>\$_FILES['file']['error']</code>	Error code associated with the upload
<code>\$_FILES['file']['size']</code>	Size of the uploaded file (in bytes)

Example:

```
print_r($_FILES['file']);
```

Uploading Files Using `move_uploaded_file()`

To store an uploaded file permanently, use the `move_uploaded_file()` function, which moves the file from its temporary location to a specified directory.

Basic File Upload Example:

```
if ($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_FILES['file'])) {
    $uploadDir = 'uploads/';
    $uploadFile = $uploadDir . basename($_FILES['file']['name']);

    if (move_uploaded_file($_FILES['file']['tmp_name'], $uploadFile)) {
        echo "File uploaded successfully.";
    } else {
        echo "File upload failed.";
    }
}
```

Setting File Restrictions

To enhance security and reliability, file uploads should be restricted based on:

Allowed File Types: Restrict uploads to specific file types using MIME types or file extensions.

```
$allowedTypes = ['image/jpeg', 'image/png', 'image/gif'];
```

```
if (!in_array($_FILES['file']['type'], $allowedTypes)) {
    echo "Invalid file type.";
    exit;
}
```

Maximum File Size: Limit file size to prevent large uploads.

```
$maxSize = 2 * 1024 * 1024; // 2MB
if ($_FILES['file']['size'] > $maxSize) {
    echo "File size exceeds the limit.";
    exit;
}
```

Preventing Malicious Uploads: To prevent malicious file uploads, store files outside the web root, rename files to avoid executing scripts, and verify file content using `finfo_file()`.

Example:

```
$finfo = finfo_open(FILEINFO_MIME_TYPE);
$mimeType = finfo_file($finfo, $_FILES['file']['tmp_name']);
if (!in_array($mimeType, $allowedTypes)) {
    echo "Potentially dangerous file detected.";
    exit;
}
finfo_close($finfo);
```

This example implements a secure profile picture upload system:

```
if ($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_FILES['profile_pic'])) {
    $uploadDir = 'uploads/';
    $file = $_FILES['profile_pic'];
    $allowedTypes = ['image/jpeg', 'image/png', 'image/gif'];
    $maxSize = 2 * 1024 * 1024;

    if (!in_array($file['type'], $allowedTypes)) {
        echo "Invalid file type.";
        exit;
    }

    if ($file['size'] > $maxSize) {
        echo "File size exceeds the limit.";
        exit;
    }

    $safeFilename = uniqid() . '_' . basename($file['name']);
    $uploadPath = $uploadDir . $safeFilename;

    if (move_uploaded_file($file['tmp_name'], $uploadPath)) {
        echo "Profile picture uploaded successfully.";
    } else {
        echo "Error uploading file.";
    }
}
```

3.5. Preventing Common Security Issues

Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) is a vulnerability that allows attackers to inject malicious scripts into web pages viewed by users. This can lead to various attacks, including session hijacking, redirection to malicious sites, or unauthorized actions on behalf of users.

To prevent XSS, always sanitize and escape user input before displaying it in HTML. Use the `htmlspecialchars()` function to convert special characters into their HTML entities, preventing script execution.

Example:

```
$input = "<script>alert('XSS Attack');</script>";
echo htmlspecialchars($input, ENT_QUOTES, 'UTF-8');
// Output: &lt;script&gt;alert('XSS Attack');&lt;/script&gt;
```

Input: The variable `$input` contains a script tag that would trigger a JavaScript alert if executed in a browser.

`htmlspecialchars()`: is used to convert special characters to HTML entities, which helps prevent Cross-Site Scripting (XSS) attacks. By escaping these characters, it ensures that user input is safely rendered in HTML.

- `ENT_QUOTES` ensures both double and single quotes are converted.
- `'UTF-8'` specifies the character encoding.

```
&lt;script&gt;alert('XSS Attack');&lt;/script&gt;
```

This prevents the browser from interpreting it as an actual script, thus mitigating the XSS vulnerability.

3.6. Regular expressions

Regular expressions (regex) are powerful tools for pattern matching and manipulation in strings. They can be used for validation, searching, and replacing text. Regular expressions are a versatile tool for string manipulation and validation.

Basics of Regular Expressions

Metacharacters: Special characters that have specific meanings in regex.

Examples:

.	Matches any single character (except newline)
*	Matches zero or more occurrences
+	Matches one or more occurrences
?	Matches zero or one occurrence
[]	Character sets

<code>^</code>	Start of string
<code>\$</code>	End of string
<code>\</code>	Escape character

Character Classes: Define sets of characters to match.

<code>[a-z]</code>	Matches any lowercase letter
<code>[0-9]</code>	Matches any digit
<code>[^a-z]</code>	Matches any character not in the set

Predefined character classes:

<code>\d</code>	Digits
<code>\D</code>	Non-digits
<code>\w</code>	Word characters (letters, digits, underscore)
<code>\W</code>	Non-word characters
<code>\s</code>	Whitespace
<code>\S</code>	Non-whitespace

Quantifiers: Specify how many times a character or group should occur.

<code>{n}</code>	Exactly n times
<code>{n,}</code>	n or more times
<code>{n,m}</code>	Between n and m times

Modifiers (Flags): Modify the behavior of the regex.

<code>i</code>	Case insensitive
<code>m</code>	Multiline
<code>s</code>	Dotall (dot matches newlines)
<code>u</code>	UTF-8

Common Use cases

Email Validation	<code>/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}\$/</code>
URL Validation	<code>/^(https? ftp):\/\/[^\s/\$.?\#].*\$/</code>
Phone Number Validation	<code>/^\+?[1-9]\d{1,14}\$/</code>
Password Strength Check	<code>/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@\$!%*?&])[A-Za-z\d@\$!%*?&]{8,}\$/</code>
Date Validation (YYYY-MM-DD)	<code>/^\d{4}-\d{2}-\d{2}\$/</code>

preg_match()

`preg_match()` is a function in PHP used for performing regular expression matching. It checks if a given string matches a specified pattern. This is particularly useful for validating input, searching for specific formats, or extracting information from strings.

Syntax

```
int preg_match(string $pattern, string $subject, array &$matches = null)
```

\$pattern: The regular expression pattern to search for. This pattern should be enclosed in delimiters (commonly slashes /).

\$subject: The string to search within.

\$matches (optional): If provided, this variable will be filled with the matches found in the subject string.

Return Value

- Returns 1 if the pattern matches the subject.
- Returns 0 if there is no match.
- Returns FALSE if an error occurred.

Example: validate an email address

```
<?php
$email = "test@example.com";
$pattern = '/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/';

if (preg_match($pattern, $email)) {
    echo "Valid email address.";
} else {
    echo "Invalid email address.";
}
?>
```

Example: validate phone number

```
<?php

// Regular expression pattern for validating Ethiopian phone numbers
$phoneNumber = "+251909090707";

$pattern = '/^\+251[0-9]{9}$/';

if (preg_match($pattern, $phoneNumber)) {
    echo "Valid phone number.";
} else {
    echo "Invalid phone number.";
}
```